ChatGPT

Upgrade Share ···

RAG_LAYOUT_REVIEW....
File

this is my current project layout

I want to somehow reduce:
1. latency
2. less data being available, and data never being enough (data thirsty)
3. Some wrong nodes, fetch wrong data, with which the LLM answers might be a bit too data affected

You've actually built a *very solid, production-grade RAG pipeline* — the issues you're seeing are not bugs, they're **natural failure modes of advanced RAG systems**. The good news: all three of your problems are fixable with targeted design changes (not a rewrite).

Let's go straight at your 3 concerns with **practical, high-impact fixes**.

🔴1) Latency is too high

Root cause (from your layout)

Latency is stacking from:

- Query rewriting (multi-variant)
- Parallel retrieval (multiple fetchers)
- Reranking (cross-encoder)
- Sentence compression (embedding-heavy)
- Large-context synthesis

👉You've built a **"max-quality pipeline"**, not a **"fast pipeline."**

✅Fix: Introduce adaptive pipeline execution (critical)

Instead of always running full pipeline:

🔁Add a *fast-path vs deep-path router*

```
Simple query → FastRAG only → top-k → direct synthesis
Complex query → full pipeline
```

Example:

Query Type	Pipeline
"What is TCP?"	❌no rewrite, ❌no graph, ❌no rerank
"Compare TCP vs QUIC performance under loss"	✅full pipeline

🔧Specific optimizations

1. Kill unnecessary query variants

- Reduce from N variants → 2 max
- Or:

Python Run

```python
if confidence_high(query):
    skip_rewrite()
```

2. Early stopping in retrieval

Instead of:

```
run all fetchers → merge
```

Do:

```
if fast_rag confidence > threshold:
    skip hybrid + graph
```

3. Replace cross-encoder rerank (big win)

Cross-encoders are **latency killers**

👉Replace with:

- bi-encoder similarity
- or lightweight reranker (like bge-reranker-base on GPU)

4. Kill sentence-level compression (or gate it)

This is one of your biggest hidden costs:

- embedding every sentence = 💀

👉Replace with:

- chunk-level scoring only
- OR:

```
if context < threshold:
    skip compression
```

5. Reduce prompt size aggressively

Instead of:

```
top 20 chunks → compress → LLM
```

Do:

```
top 5–8 chunks max
```

6. Move embeddings off CPU (massive win)

Right now:

```
embedding_device: cpu
```

👉This alone can give **5–20x speedup**

🟡2) “Data is never enough” (data hunger problem)

This is a **classic RAG illusion**:

> No matter how much data you add, recall still feels insufficient.

Root causes

From your setup:

- Hard caps (`max_rows`)
- Static datasets (Wikipedia snapshot)
- No dynamic retrieval (web/live sources)
- Query mismatch vs dataset distribution

✅Fix: Shift from “more data” → “better retrieval”

🔥1. Add query-aware retrieval expansion

Instead of:

```
embed(query) → search
```

Do:

```
query → intent → targeted retrieval strategy
```

Example:

Query	Retrieval

Query	Retrieval
coding	StackOverflow-heavy
factual	Wikipedia
reasoning	hybrid + dense

🔥2. Add on-demand retrieval (huge unlock)

Instead of relying only on your static DB:

- Add:
  - web search fallback
  - API connectors
  - domain-specific sources

```
if low confidence:
    trigger external retrieval
```

🔥3. Improve chunking (very important)

Your current chunking likely causes:

- context fragmentation
- missed relevance

👉Try:

- semantic chunking (not fixed size)
- overlap tuning (10–20%)
- hierarchical chunks (section → paragraph)

🔥4. Add query coverage feedback loop

Log:

- failed queries
- low-confidence answers

Then:

- selectively expand dataset
- not blindly scale data

🔵3) Wrong nodes / bad retrieval contaminating answers

This is the **most important problem** (accuracy killer).

Root cause

Your pipeline currently:

```
retrieve → merge → rerank → compress → synthesize
```

👉BUT:

- No **strict filtering**
- No **grounding enforcement**
- LLM trusts noisy context

✅Fix: Add retrieval trust control

🔥1. Introduce a relevance threshold (mandatory)

Before synthesis:

Python Run

```python
if score < threshold:
    DROP chunk
```

👉Do not pass low-confidence chunks to LLM

🔥2. Add agreement filtering (very powerful)

Keep only facts supported by multiple sources:

```
if fact appears in 2+ chunks:
    keep
else:
    drop or downweight
```

🔥 3. Use source diversity penalty

Avoid:

- same doc dominating results

🔥 4. Add “abstain mode” (critical for quality)

Instead of hallucinating:

```
if context confidence low:
    return "Not enough information"
```

🔥 5. Make synthesis stricter

In your prompt:

```
- Only answer using provided sources
- If conflicting info → mention uncertainty
- If insufficient data → say so
```

🔥 6. Add lightweight verification step (optional high-end)

After generation:

```
answer → re-check against retrieved chunks
```

🧠The Real Insight (Important)

Right now your system is:

> "Maximize information → then trust LLM to fix it"

That's why you see:

- latency ↑
- noise ↑
- data hunger ↑

🔄What you should move toward:

> "Minimize noise → give only high-confidence info → constrain LLM"

🚀Priority Fix Order (Do THIS first)

If you only do a few things, do these:

1. ⚡ Move embeddings to GPU

→ instant big win

2. 🔥Reduce pipeline depth

- skip rewrite for simple queries
- limit fetchers

3. 🧹 Add strict filtering before synthesis

→ biggest accuracy gain

4. ✂️Remove sentence-level compression

→ huge latency reduction

5. 📉Reduce top-k context

→ faster + better answers

If you want next step

I can:

- redesign your pipeline into a **2-tier fast + deep architecture**
- or give you a **concrete refactored flow diagram**
- or suggest **exact code-level changes per module**

Just tell me 👍